Machine Learning for Artificial Intelligence Engineering, Winter Term 2026/27, Exercise Sheet P1
Mathematical Preliminaries: p-Norms, Linear Algebra and Optimization.

Alexander Mayr
alexander.mayr@tugraz.at

Points to achieve:	15 Points
Deadline:	15.10.2026 23:59 (strict, no late submissions allowed)
Review Session:	16.10.2026, in person (see General remarks)
Hand-in procedure:	Work in pairs (your TeachCenter group). **Exactly one** team member submits **all Python files and a report (PDF)** to the TeachCenter. Do not rename the Python files. Do not zip them. Do not upload the `plots` folder.
Generative AI:	Allowed – but you must be able to explain every part of your solution without it (see General remarks).
Plagiarism:	If detected, 0 points on the entire exercise sheet for all parties involved. If this happens twice, we will grade the group with "Ungültig aufgrund von Täuschung"
Course info:	TeachCenter

Contents

1 p-Norms [5 points] **3**

2 Linear Algebra [5 points] **3**

3 Optimization [5 points] **4**

General remarks

Your submission will be graded based on:

- Correctness (Is your code doing what it should be doing?)
- Your derivations (Show the intermediate steps. Typeset and legible handwritten scans are both fine.)
- The depth of your interpretations (Usually, only a couple of lines are needed.)
- The quality of your plots (Is everything clearly readable/interpretable? Are axes labeled? ...)
- Your report starts with the **cover letter** (names and matriculation numbers of both team members, see the introductory practical).
- Your submission must run with Python 3.11 (or newer) and the package versions listed in `requirements.txt` (e.g., run `pip install -r requirements.txt` in a fresh Python environment).

Since we run automated tests, it is crucial that you keep the following in mind:

- **Do not add any additional `import` statements** anywhere in the code.
- **Do not modify the function signatures** of the skeleton functions
  - i.e., do not edit the function names and inputs
- **Do not change the random seed** that is set in the code skeleton.

Code Skeleton

The code skeleton consists of `main.py`, `norms.py`, `linear_algebra.py`, `optimization.py`, `plot_utils.py` and `requirements.txt`. Fill in all `TODO`s in the first four files; `plot_utils.py` contains all plotting functions and does not need to be edited. Run `python main.py` to run all tasks, or, e.g., `python main.py 3` to run a single task. All plots are saved to the folder `plots`.

NumPy

- This sheet only uses `NumPy` and `Matplotlib` – no `pandas`, no `scikit-learn`, no `SciPy`.
- Some tasks ask you to implement an operation that NumPy already provides (the p-norm, the dot product and the matrix product). In these cases, **do not call the corresponding NumPy routine inside your function** – `main.py` uses it only to verify your result. Element-wise arithmetic (`+`, `-`, `*`, `/`, `**`), `np.sum`, `np.sqrt`, `np.abs`, `np.max`, indexing and loops are always allowed.
- Compare floating point results with `np.allclose`, never with `==`. Remember that `*` is the *element-wise* product and `@` the matrix product. If something looks wrong, check the `.shape` of your arrays first.

Use of Generative AI

The use of generative AI (coding assistants, LLMs) is **explicitly allowed** for this and every other exercise sheet. However, if your group is drawn in the review session, you must be able to **explain and defend every part of your solution** – code, derivations and interpretations – **without AI assistance**. All tasks on this sheet can be solved with the slides of the practical session *Mathematical Preliminaries* alone.

At the end of your report, **include a short paragraph that details your usage of generative AI**: (1) which tool(s) you used, (2) how it helped (e.g., debugging code, explaining a derivation), and (3) its limitations (e.g., generated code or derivations that were wrong and had to be fixed).

Review Session

In the review session on 16.10.2026, 3–6 groups are drawn at random from the pool of available groups to present and discuss their solutions. Your group is in the pool unless you state in your submission that you are **not available** ("Not present"), together with a reason. To receive points for all exercise sheets, your group must be available at least twice over the semester. If a group that is available is drawn and does not present, the sheet is graded with 0 points.

1 p-Norms [5 points]

In machine learning, a single data point is usually represented as a vector $\mathbf{x} = (x_1, \ldots, x_D)^T \in \mathbb{R}^D$. Many methods (e.g., K-Nearest Neighbours or K-Means) compare data points by their distance $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$, which requires a *norm*. The p-norm of $\mathbf{x}$ is defined as

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{D} |x_i|^p \right)^{1/p} \quad (p \geq 1), \qquad \|\mathbf{x}\|_\infty = \max_{i=1,\ldots,D} |x_i|. \tag{1}$$

Tasks:

1. **[2 points]** Show that Equation (1) with $p = 2$ yields the Euclidean norm, i.e., $\|\mathbf{x}\|_2 = \sqrt{x_1^2 + \cdots + x_D^2} = \sqrt{\mathbf{x}^T\mathbf{x}}$. Then compute $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_2$ and $\|\mathbf{x}\|_\infty$ for $\mathbf{x} = (3, -4)^T$. Show all intermediate steps.
2. **[2 points]** Implement `p_norm(x, p)` in `norms.py`. It must also handle `p = np.inf`. `main.py` compares your function with `np.linalg.norm`; report the outcome of these checks. Print the three norms of $\mathbf{x} = (3, -4)^T$ in `task_1` and compare them with your results from Task 1.1.
3. **[1 point]** `main.py` plots $\|\mathbf{x}\|_p$ over $p = 1, 2, \ldots, 20$ for the three vectors $\mathbf{x}_1 = (3, -4)^T$, $\mathbf{x}_2 = (1, 1, 1, 1)^T$ and $\mathbf{x}_3 = (0, 2, 0)^T$. Include the plot in your report and describe what you observe. Which value does $\|\mathbf{x}\|_p$ approach for large p? Why does $\|\mathbf{x}_3\|_p$ not depend on p?

2 Linear Algebra [5 points]

The *rank* of a matrix is the maximum number of linearly independent columns (or rows). A square matrix $C \in \mathbb{R}^{N \times N}$ is invertible if and only if $\text{rank}(C) = N$. The dot product of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^D$ and the product of two matrices $A \in \mathbb{R}^{M \times N}$ and $B \in \mathbb{R}^{N \times K}$ are

$$\mathbf{x}^T\mathbf{y} = \sum_{i=1}^{D} x_i y_i, \qquad (AB)_{ij} = \sum_{k=1}^{N} a_{ik} b_{kj} = \mathbf{a}_{i:}^T \mathbf{b}_{:j}, \tag{2}$$

i.e., the entry $(AB)_{ij}$ is the dot product of the i-th row of A and the j-th column of B.

Tasks:

1. **[2 points]** Compute the rank of the matrix

$$C = \begin{pmatrix} 1 & 2 & 0 \\ 2 & 1 & 3 \\ 3 & 3 & 3 \end{pmatrix}.$$

   Show all intermediate steps (e.g., write one column as a linear combination of the others, or use Gaussian elimination). Is C invertible? Justify your answer. Check your result with `np.linalg.matrix_rank` in `task_2`.
2. **[2 points]** Implement your own versions of `np.dot` and `np.matmul`: the functions `dot(x, y)` and `matmul(A, B)` in `linear_algebra.py`, using Equation (2). Your `matmul` must call your `dot` once for every entry of AB. Do not use `np.dot`, `np.matmul` or `@` inside these functions.
3. **[1 point]** `main.py` tests your functions on the examples from the lecture. Extend the tests in `task_2`: compare your `matmul` with `np.matmul` for random matrices of shape 3×5 and 5×2, and check, for two random 3×3 matrices A and B, whether $AB = BA$ and whether $(AB)^T = B^T A^T$. Report the results.

3 Optimization [5 points]

Learning a model means minimizing a loss function. Gradient descent starts at an initial point $x^{(0)}$ and repeats

$$x^{(t+1)} = x^{(t)} - \eta\, f'(x^{(t)}) \qquad \text{while} \quad |f'(x^{(t)})| > \varepsilon, \tag{3}$$

where $\eta > 0$ is the *learning rate*. (For functions $f : \mathbb{R}^D \to \mathbb{R}$, the derivative f' is replaced by the gradient ∇f.) In this task, we minimize the function

$$f(x) = x^4 - 2x^2. \tag{4}$$

Tasks:

1. **[2 points]** Compute the derivative $f'(x)$ and all stationary points of f (i.e., all x with $f'(x) = 0$). Show all intermediate steps. Which stationary points are minima and which are maxima? Is f convex?
2. **[1 point]** Implement `f` and `df` (your derivative from Task 3.1) in `optimization.py`. `main.py` plots f and f'. Include the plot in your report and explain how the sign of $f'(x)$ is related to whether f increases or decreases.
3. **[2 points]** Implement `gradient_descent(grad, x0, learning_rate, eps, max_iters)` following Equation (3). It stops when $|f'(x^{(t)})| \leq \varepsilon$ or after `max_iters` steps and returns all iterates. `main.py` runs gradient descent (i) with $\eta = 0.05$ from the initial points $x^{(0)} \in \{-1.5, -0.3, 0, 0.3, 1.5\}$ and (ii) from $x^{(0)} = 1.5$ with the learning rates $\eta \in \{0.01, 0.05, 0.15, 0.3\}$. Print the number of iterations and the final iterate for every run and include the plots. Explain where each run of (i) ends and why, in particular for $x^{(0)} = 0$. For (ii), describe the effect of the learning rate.